



Operadores Relacionais e Lógicos

Curso: Programador de Sistemas

Senac Centro | Noite

O que vimos até aqui?

- Variáveis (`let` e `const`) servem para guardar informações na memória.
- Tipos de dados: `String` (texto), `Number` (números), `Boolean` (`true/false`), `null` e `undefined`.
- Operadores aritméticos: `+`, `-`, `*`, `/`, `**` (potência) e `%` (resto da divisão).

Álgebra Booleana e Lógica

Toda tomada de decisão se baseia na **Lógica Binária**:

- Criada por George Boole no século XIX.
- Apenas dois estados possíveis: Verdadeiro (`true` / `1`) ou Falso (`false` / `0`).
- No computador, tomamos decisões avaliando se condições são verdadeiras ou falsas.

Expressões no Dia a Dia

Antes de programar, nós já pensamos em expressões lógicas:

- "A temperatura é maior que 30°C?" → `true` ou `false` .
- "O cliente tem saldo na conta?" → `true` ou `false` .
- "O usuário digitou a senha correta?" → `true` ou `false` .

Como o programa interage?

Até agora, nossos códigos usam valores fixos (definidos diretamente no código).

Como podemos fazer o programa **perguntar** algo para o usuário no terminal?

No Node.js, usamos a biblioteca: **prompt-sync** .

Instalando o `prompt-sync`

No terminal da sua pasta do projeto, execute:

```
npm install prompt-sync
```

Isso criará a pasta `node_modules` e baixará a biblioteca necessária.

Importando e Lendo Textos

No topo do seu arquivo JavaScript, adicione a importação:

```
// Importa e inicializa a biblioteca
const prompt = require('prompt-sync')();

// Lê um texto digitado pelo usuário
let nome = prompt("Qual o seu nome? ");
console.log(`Olá, ${nome}!`);
```

12 34 Lendo Números (Conversão)

Tudo que é lido pelo terminal vem como **String** (texto). Para fazer cálculos, precisamos converter para **Number**:

```
const prompt = require('prompt-sync')();

let idadeText = prompt("Sua idade: ");
let idade = Number(idadeText); // Converte!

// Ou de forma direta:
let anoNascimento = Number(prompt("Ano de nascimento: "));
```


Prática Rápida: Dobro

Crie um arquivo `dobro.js` . Peça para o usuário digitar um número, calcule o dobro dele e exiba no terminal.

```
const prompt = require('prompt-sync')();  
  
let numero = Number(prompt("Digite um número: "));  
let dobro = numero * 2;  
  
console.log(`O dobro de ${numero} é ${dobro}`);
```

Operadores Relacionais

Servem para **comparar** dois valores. O resultado de uma comparação é sempre um valor Booleano (`true` ou `false`).

- `>` (Maior que) | `<` (Menor que)
- `>=` (Maior ou igual) | `<=` (Menor ou igual)
- `==` (Igualdade de valor)
- `===` (Igualdade estrita: valor E tipo de dado)
- `!=` (Diferente) | `!==` (Diferente estrito)

A Pegadinha do `==` vs. `===`

O JavaScript tenta ser "legal" e faz conversão implícita com `==`:

- `"5" == 5` → `true` (Ele converte o texto para número e compara).
- `"5" === 5` → `false` (Os tipos são diferentes: String vs. Number).

Dica de ouro: Use sempre o comparador estrito `===` e `!==` para evitar bugs invisíveis.

E a Diferença? `!=` vs. `!==`

A mesma pegadinha da conversão automática acontece quando testamos se valores são diferentes:

- `"5" != 5` → `false` (Ele converte os tipos e diz que eles *não* são diferentes).
- `"5" !== 5` → `true` (Diferença estrita: os tipos são String e Number, logo eles *são* diferentes).

Dica: Prefira sempre o operador estrito `!==` para manter seu código seguro.

Prática 1: Estoque Mínimo

Verificar se um item precisa de reposição no estoque lendo os dados de forma interativa.

```
const prompt = require('prompt-sync')();

// Lendo do terminal
let estoqueAtual = Number(prompt("Estoque Atual: "));
let estoqueMinimo = Number(prompt("Estoque Mínimo: "));

// Comparação lógica
let precisaRepor = estoqueAtual ≤ estoqueMinimo;

console.log("Precisa repor?", precisaRepor);
```

🤝 0 Operador `&&` (AND / E)

Exige que **todas** as condições sejam verdadeiras para retornar `true`.

- *Exemplo:* Para entrar na balada, você precisa ser maior de 18 anos E ter o ingresso comprado.

```
let idade = 20;  
let temIngresso = true;  
  
let podeEntrar = (idade ≥ 18) && temIngresso;  
console.log(podeEntrar); // true
```



Prática 2: Sistema de Login

Validar se as credenciais digitadas pelo usuário batem com as do sistema.

```
const prompt = require('prompt-sync')();

const usuarioSalvo = "admin";
const senhaSalva = "senac123";

// Lendo do usuário
let usuarioDigitado = prompt("Usuário: ");
let senhaDigitada = prompt("Senha: ");

let loginSucesso = (usuarioDigitado === usuarioSalvo) && (senhaDigitada === senhaSalva);
console.log("Login OK?", loginSucesso);
```

O Operador `||` (OR / OU)

Basta que uma das condições seja verdadeira para retornar `true`.

- *Exemplo:* Você ganha frete grátis se for cliente VIP OU se a sua compra passar de R\$ 150.

```
let isVip = false;  
let valorCompra = 200;  
  
let freteGratis = isVip || (valorCompra ≥ 150);  
console.log(freteGratis); // true
```


Prática 3: Promoção Frete Grátis

Verifique se a compra deu frete grátis baseando-se no valor ou no status de cliente VIP.

```
const prompt = require('prompt-sync')();

let valorCompra = Number(prompt("Valor da Compra: R$ "));
let respostaVip = prompt("Você é cliente VIP? (s/n): ");
let isClienteVip = (respostaVip === "s" || respostaVip === "S");

let ganhouFrete = (valorCompra ≥ 150) || isClienteVip;
console.log("Ganhou frete grátis?", ganhouFrete);
```

O Operador **!** (NOT / NÃO)

Ele simplesmente inverte o estado lógico de um booleano.

- `!true` vira `false`
- `!false` vira `true`

```
let temRestricaoCPF = false;  
let cpfLimpo = !temRestricaoCPF;  
  
console.log("CPF limpo?", cpfLimpo); // true
```

Precedência de Operadores

Quando misturamos tudo, a ordem que o JavaScript avalia a expressão é:

1. **Aritméticos:** (`*` , `/` , `+` , `-`)
2. **Relacionais:** (`>` , `<` , `===` , `!==`)
3. **Lógicos:** (`!` , `&&` , `||`)

Dica: Use parênteses `()` para forçar a ordem que você deseja!

Desafio Final: Crédito Bancário

Um cliente é pré-aprovado para empréstimo se:

- Tiver idade maior ou igual a 18 anos.
- Tiver renda mensal superior a R\$ 3.000.
- **NÃO** tiver restrição no CPF (`temRestricaoCPF` deve ser `false`).

```
const prompt = require('prompt-sync')();

let idade = Number(prompt("Idade: "));
let renda = Number(prompt("Renda Mensal: R$ "));
let respostaRestricao = prompt("Tem restrição no CPF? (s/n): ");
let temRestricaoCPF = (respostaRestricao === "s" || respostaRestricao === "S");

// Monte a expressão combinando relacionais, AND e NOT
let aprovado = (idade ≥ 18) && (renda > 3000) && (!temRestricaoCPF);
console.log("Crédito aprovado?", aprovado);
```



Conclusão

Agora você é capaz de fazer o computador tomar decisões complexas combinando comparações lógicas!

Próxima Aula: Estruturas de Decisão (`if`, `else if`, `else`) para controlar o fluxo de execução.